

Empowering Physical Security Systems with AI: Leveraging LLMs and RAG for Seamless API Integration

Chris Peckham, CPP, PSP

Ollivier Corporation

Abstract

In this session, we will show how the use of Large Language Models (LLMs) can enhance physical security systems through Retrieval-Augmented Generation (RAG) and API function calling. By integrating LLMs with access control systems, video management systems, and endpoints, we enable AI-driven interactions for tasks such as querying events, modifying settings, and retrieving video footage. Attendees will learn about LLM foundations, the use of RAG for real-time knowledge integration, and how to execute structured actions via APIs. This session will also cover architectural approaches, security considerations, and best practices for integrating AI with physical security technologies.

Learning Objectives

Learning Objective #1: Explain foundational concepts of LLMs, including chatbots, RAG, and function calling for enabling natural language API interactions.

Learning Objective #2: Identify API capabilities of physical security systems and practical LLM use cases for automation and control.

Learning Objective #3: Design secure architectures integrating LLMs with security APIs, covering real-time function calls, authentication, and error handling

The Problem We're Trying to Solve

- Every physical security platform speaks a different language: REST, SOAP/XML, proprietary SDKs, PS Tools
- Different auth, different terminology, different quirks
- Operators

A Quick Primer — LLM, RAG, MCP, API

LLM — Large Language Model

The reasoning engine — reads text, understands intent, generates a response. On its own it doesn't know anything about your specific systems.

RAG — Retrieval-Augmented Generation

Before answering, relevant docs/notes are looked up and handed to the LLM as extra context — grounds it in facts it wasn't trained on.

MCP — Model Context Protocol

A standard way to expose “tools” (functions) an LLM can call — structured requests/responses, not free-form text guessing.

API — Application Programming Interface

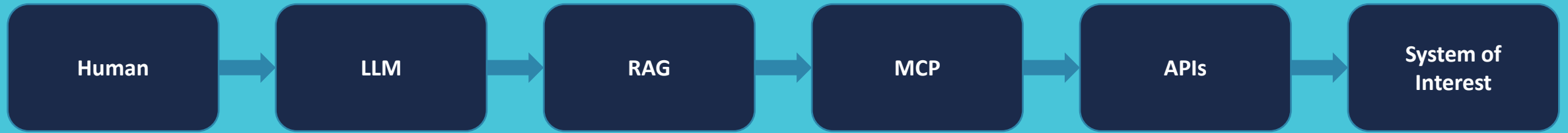
The actual interface the security system exposes — REST, SOAP/XML, PowerShell, etc. This is what really talks to the hardware/software.

How they fit together:

LLM interprets intent → RAG supplies grounding context → LLM picks an MCP tool → MCP calls the real API → result flows back up

Learn more: Model Context Protocol spec (modelcontextprotocol.io) • Anthropic docs (docs.claude.com) • Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”, 2020 ([arXiv:2005.11401](https://arxiv.org/abs/2005.11401)) • OWASP Top 10 for LLM Applications 2026 (genai.owasp.org) • OWASP API Security Top 10 (api-security.owasp.org)

The Concept

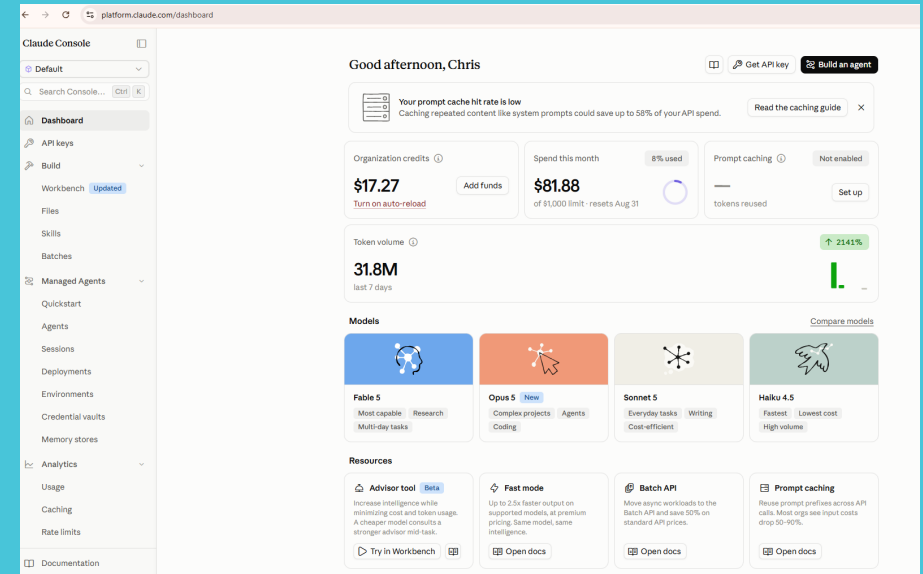


One continuous pipeline, left to right: a plain-English question goes in on the left; a real action or answer from a real physical security system comes out on the right.

Piece	Role
LLM	Understands intent, generates responses
RAG	Gives the LLM system-specific knowledge it wasn't trained on
MCP	Structured mechanism for exposing tools to the AI
API	Interface to the physical security platform
System of Interest	Remains the authoritative system of record and execution

The LLM Is Swappable

- Claude was used throughout this work for quality/convenience. Nothing here is Claude-specific.
- MCP is model-agnostic: any model with reliable tool calling can sit behind this exact architecture.
- This project already self-hosts half the AI stack: every RAG pipeline's embedding model runs 100% locally, no network calls, no paid API. Only the reasoning LLM is currently hosted.



Actual Claude API usage/cost for this project

Self-Hosting Considerations

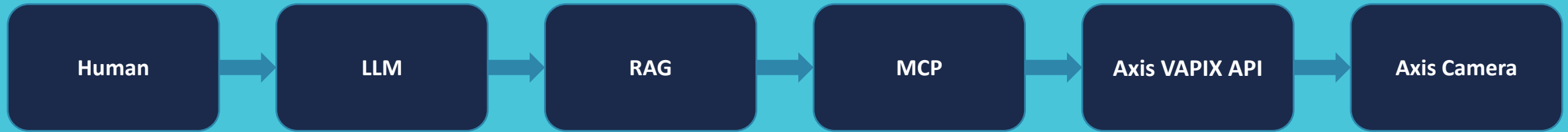
If self-hosting the reasoning model too, plan for:

- Tool-calling reliability varies a lot across open-weight models, Measure it, don't assume
- Instruction-following under edge cases. The confirmation-gate/dry-run safety model depends on it
- Context window and long-context recall. RAG chunks + tool schemas + history need real headroom
- Hardware sizing for concurrent operators, not just single-user demo latency
- Quantization tradeoffs. Re-verify tool-calling accuracy after quantizing
- You now own patch/update timing. No silent vendor swaps, no vendor patching for you either

Real options if you want to try this yourself: Ollama or llama.cpp running Llama 3.1 8B / Qwen3 4B for general chat+tool-calling, or Google's edge-tuned Gemma E2B/E4B (E4B ≈ 5GB at 4-bit — fits under 8GB RAM). Most laptops from the last ~5 years manage ~10 tokens/sec on CPU alone — no GPU required to start experimenting. (source: How-To Geek, "Excellent Local LLM Projects")

Real Systems

Case Study #1: Axis Camera



- 46 tools: streaming, snapshot, PTZ, overlays, clock, light, user management — most live-verified against real hardware



Live snapshot, AXIS P3247-LVE, warehouse (2026-08-11)

****You:**** Using the axis-camera tools directly (not Milestone's), take a snapshot from the camera right now and tell me what you can see in it, in two to three sentences.

****Claude:**** I'll search for the axis-camera tools. That search surfaced only Milestone tools. Let me look at what the axis-camera server actually exposes. I found the axis-camera snapshot tool. Let me load its schema and call it. Snapshot captured via `axis-camera`'s `get\_snapshot`:

The camera is mounted high in a corner of a small office or lab room, angled steeply upward so most of the frame is white ceiling with a lit fluorescent tube fixture. On the far wall there's a whiteboard with faint hand-drawn diagrams and notes, and to the right a dark gray two-drawer filing cabinet sits below a wall-mounted HVAC/AC unit, with a teal-colored device or camera and some boxes stacked on top of it. In the lower left foreground, a bald man wearing glasses and a dark shirt is partially in frame and motion-blurred, apparently moving past the camera near a wooden door, with an office chair just visible at the bottom.



(19.7s)

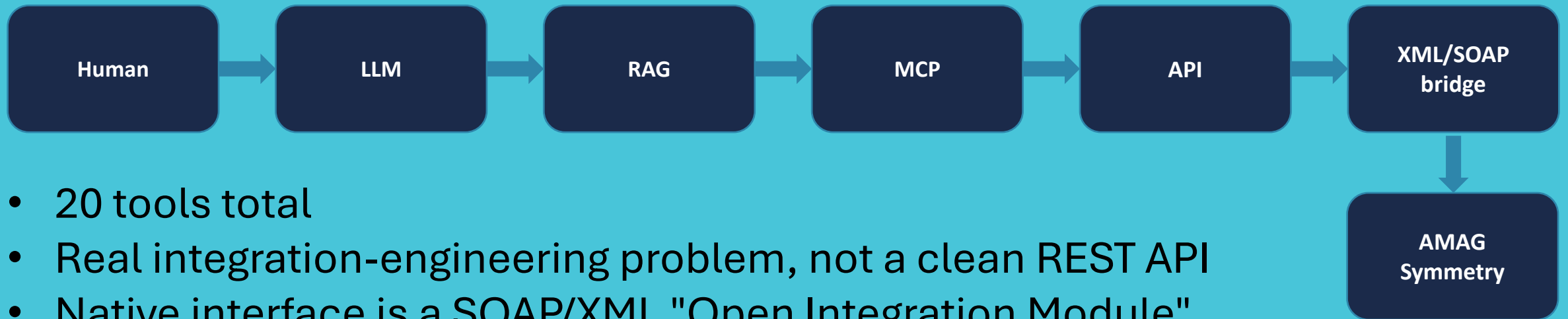
Grab a snapshot from an Axis camera and describe the scene

Axis: What Does RAG Know Here?

- Corpus: VAPIX docs + this project's own live-verified findings (confirmed device behavior, not just spec text)
- Retrieval: hybrid BM25 + embedding search, local, no paid APIs

The LLM doesn't need to memorize VAPIX. It retrieves, then selects a tool and populates parameters.

Case Study #2: AMAG Symmetry



- 20 tools total
- Real integration-engineering problem, not a clean REST API
- Native interface is a SOAP/XML "Open Integration Module"
- Created a .NET REST wrapper sitting in front of it
- Blazor web UI for interaction and testing
- LLM can now interface with Symmetry
- AI doesn't eliminate integration engineering; it changes where that layer lives

AMAG: The Legacy System Problem

- Legacy systems, proprietary interfaces, undocumented behaviors, systems never designed for AI
- An AI-facing abstraction layer in front of the legacy adapter, not direct LLM ↔ legacy wiring
 - AMAG's cardholder search enum declares SEARCH_BY_LF_READER/READERGROUP/FLOORGROUP (a reverse "who has access to X" lookup)
 - live-testing against known-good data shows this installation's API doesn't functionally support it, regardless of what the schema says
 - RAG-sourced, live-verified notes catch that, a spec alone doesn't

AMAG: What Does RAG Know Here?

- Corpus: this project's own docs — README.md, docs/ROADMAP.md, docs/live-verification-checklist.md — not AMAG's own vendor documentation, which is sparse
- 174 chunks indexed — smallest corpus of the three demos, deliberately: this is operational knowledge (what actually works on this installation), not a full API reference dump
- Embedding: all-MiniLM-L6-v2, quantized ONNX (~23MB, vendored, not downloaded at runtime), run via ONNX Runtime. Zero network calls at query time (<https://huggingface.co/onnx-models/all-MiniLM-L6-v2-onnx>)
- Storage/retrieval: one SQLite file (chunks + vectors together), hybrid BM25 + vector search fused via Reciprocal Rank Fusion

****You:**** What was the door/badge activity at company 67 on Sunday 9 Aug 2026? Summarize the comings and goings.

****Claude:**** I'll search for the relevant access control tools. I have the activity search tool. Let me also grab system info to confirm what company 67 actually is. The result was too large and got spilled to a file. I need file-reading tools to analyze it. No general-purpose file or shell tools exist here, so I can't read the spilled files directly. Let me check the MCP resource readers, and meanwhile re-run the query in narrower time windows so the results fit inline. Company 67 is ****Kohler, Schuppe and Metz****. Fetching the rest of the day in chunks:## Door/badge activity — Company 67 (Kohler, Schuppe and Metz), Sunday 9 Aug 2026

****How I pulled this:**** the full-day query was too big to return inline, so I ran it in six consecutive windows covering 00:00:00–23:59:59. All six came back with `truncated: false`, so this is the complete day, no gaps.

Headline numbers

- ****~79 credential events**** (all "Granted Access" or "Floor Access")
- ****16 distinct cardholders****
- ****5 readers/doors**** active
- ****Zero denied/rejected reads all day**** — nobody was turned away

Doors involved

| Reader | Role |

|---|---|

| 421 Kuvalis Keys – Gate 1 | perimeter gate, busiest |

| 325 Bernhard Vista – Main Entry | main building entry |

| 153 Kiehn Valley – Stairwell Door | stairwell |

| 079 Quitzon Fork – Main Entry | secondary building |

| 80265 Stark Villages – Gate 1 | outlying gate |

The comings and goings

****Overnight (01:00–06:00) — thin but not empty.**** Adalberto Ratke (card 8265452) at Bernhard Vista Main Entry 01:15, then the Kiehn Valley stairwell twice, 03:14 and 03:25. Omer Kuhic (2690823) through Gate 1 at 03:42. That's the only presence on site until the morning.

****Morning (06:00–10:00) — shift start.**** Kuhic re-enters Gate 1 06:25 → Bernhard Vista 06:41. Ratke Gate 1 06:43. Mckayla Kulas (3375631) arrives at the stairwell 06:56 and then works the building hard through 09:03 — stairwell at 06:56, 07:58, 08:01, 08:26, 09:02 with Bernhard Vista reads interleaved. Nicholas Kautzer 06:58, Lea Stracke at Quitzon Fork 08:22. Quiet gap 09:42–10:28.

****Midday (10:00–14:00) — busiest stretch.**** Bridie Corwin 10:51, Dayton Gorczany in at 11:25 (Gate 1 → Bernhard Vista), Samir O'Kon 11:57, Brennan Stoltenberg 11:58 (Gate 1 → Quitzon Fork). Kulas continues cycling Bernhard Vista at 12:02, 12:21, 13:10, then stairwell 13:36 and Quitzon Fork 13:39. Travis Tromp Gate 1 13:05, Humberto Grady Gate 1 13:33.

****Afternoon/evening (14:00–19:00).**** Bartholome Kihn in 14:21. Ansley Beer (3751893) starts at 14:42 and becomes the single most active badge of the day (****16 reads****), moving between Bernhard Vista, the stairwell, Gate 1, Stark Villages Gate 1 and Quitzon Fork. Maiya Bednar (6 reads) works a similar circuit 14:45–19:13. Gorczany out via Gate 1 → Stark Villages 15:50. Grady Gate 1 again 16:20. Alayna Hilll single read at Quitzon Fork 16:41. Kathryn Mertz Gate 1 → Stark Villages 19:06.

****Late (19:00–24:00) — busier than the morning.**** Kuhic returns and racks up reads at Quitzon Fork, the stairwell, Bernhard Vista and Gate 1 from 19:38 through 22:48 (13 reads total for the day). Gorczany back through Gate 1 20:43. Grady Gate 1 22:01, Tromp Gate 1 22:08. Beer closes the day out — Quitzon Fork 22:22/22:25, Bernhard Vista 23:07, stairwell 23:24.

****Most active badges:**** Ansley Beer (16), Mckayla Kulas (15), Omer Kuhic (13), then Gorczany and Bednar (6 each).

Two things worth your attention

****1.** Kuvalis Keys Gate 1 spent most of the day standing open.****** There are roughly 60 Pre-Held Timer → Held Open → Timer Expires cycles from about 03:42 right through to 21:41. This gate is effectively propped or repeatedly held rather than cycling normally.

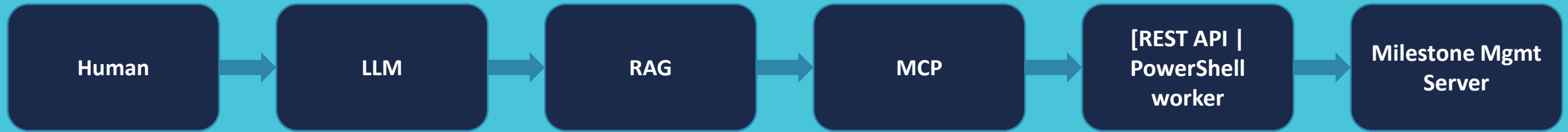
****2.** A large volume of "Door Forced" alarms that look like miscalibration, not intrusion.****** 32 at Gate 1 and 21 at the Kiehn Valley stairwell — but look at the timing: the Forced event repeatedly fires **two to eight seconds before** a perfectly valid badge grant at the same reader. Examples: 13:32:56 forced → 13:32:58 Grady granted; 22:01:00 forced → 22:01:08 Grady; 22:08:02 forced → 22:08:04 Tromp; 22:48:18 forced → 22:48:22 Kuhic. Same pattern at the stairwell (23:23:38 forced → 23:24:14 Beer granted). That signature — door contact opening a beat ahead of the reader decision, every alarm auto-resetting within seconds — points to a door contact/timing calibration problem or people pulling the door as they present the card, rather than genuine forced entry. Every one of these was followed by an Alarm Reset.

Combined, those two points mean this company's alarm log is very noisy and would mask a real forced-entry event if one occurred.

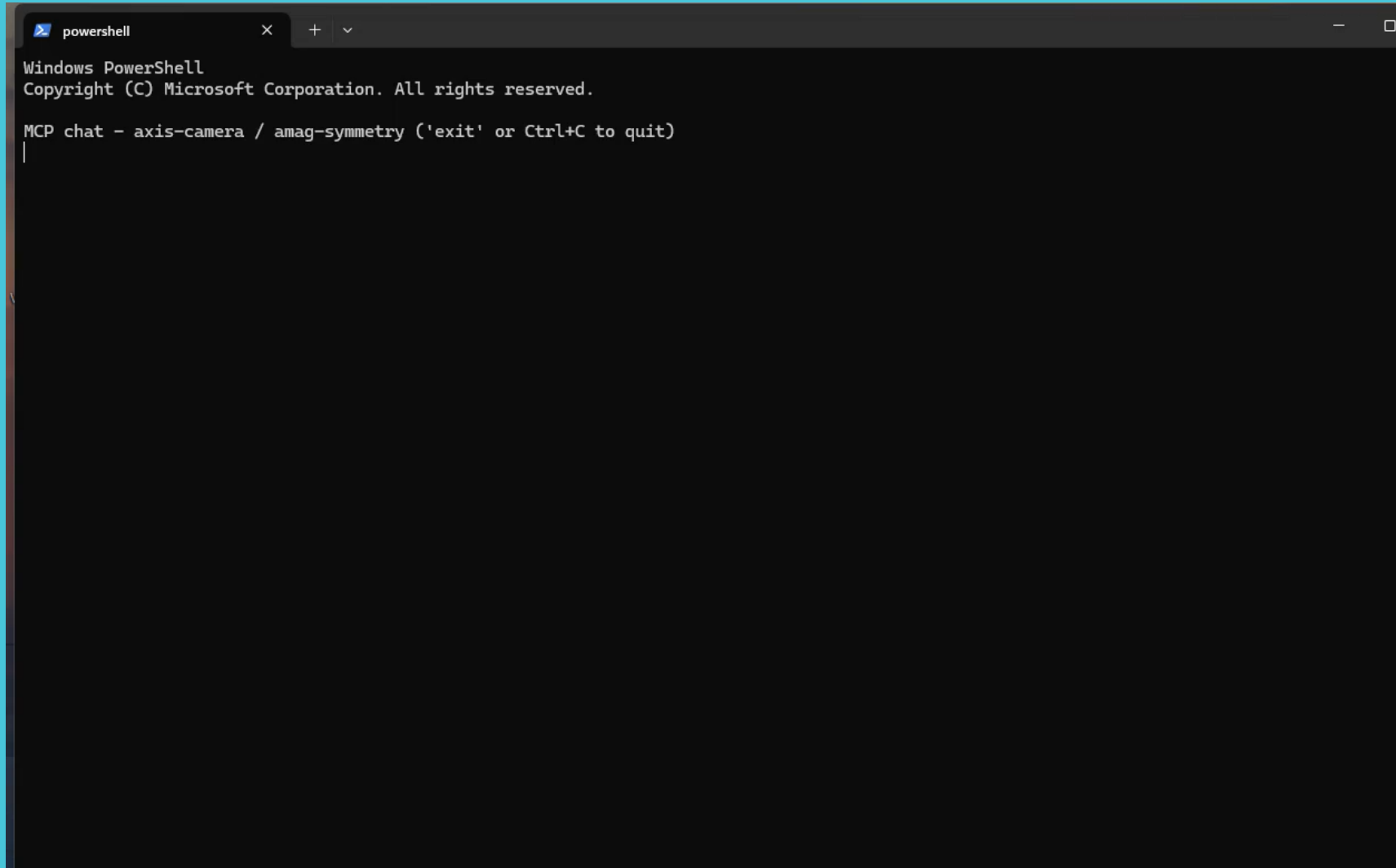
A couple of caveats: these are Symmetry's recorded local timestamps, and the readers here don't distinguish in from out, so "comings and goings" is inferred from reader sequence rather than explicit entry/exit direction. If you want to visually verify any of the Forced events at Gate 1 or the stairwell, there's an Axis camera server connected and I could pull snapshots or check what cameras cover those doors.

(121.2s)

Case Study #3: Milestone XProtect



- Milestone's own REST API doesn't cover the whole admin surface; This project uses two backends side by side: REST (~1s/call) where it covers the need, MilestonePSTools via a persistent PowerShell worker (~1-2s/call warm, ~15-20s cold start) everywhere REST falls short. Did not use SDK
- Every write tool defaults to dry run=True, reporting only what it would do.
 - Unprompted finding: asked only for a camera/license overview, the model flagged a license grace-period risk on its own. Nobody asked about licensing.
 - Add-camera dry run independently verified its own driver-number assumption and refused to fabricate a missing password.
 - Remove-camera dry run quantified the stakes (~38.7 hrs of video, no undo) before discussing execution, then proposed reversible alternatives.



```
powershell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

MCP chat - axis-camera / amag-symmetry ('exit' or Ctrl+C to quit)
|
```

Milestone camera inventory + license overview)

Milestone: What Does RAG Know Here?

- Corpus: crawled MIP VMS REST API docs + PowerShell (MilestonePSTools) cmdlet docs + a snapshotted live system state export
- 47 tools
- 1,001 chunks indexed — 600 tokens each, 80-token overlap, via LlamaIndex's SentenceSplitter
- Embedding: BAAI/bge-small-en-v1.5 (HuggingFace), runs locally. Zero network calls at query time
- Storage/retrieval: ChromaDB, vector search via the LlamaIndex framework
- This is only system of the three built on an off-the-shelf RAG framework rather than hand-rolled

Three Systems, Three Shapes

System	Tools	Integration challenge	Interface
Axis	46	Direct device API	VAPIX
AMAG	20	Legacy SOAP/XML	Custom REST→XML bridge
Milestone	47	Enterprise VMS breadth, REST doesn't cover everything	REST + MilestonePSTools (PowerShell) hybrid

AI integration isn't just one problem.

The shape of the integration layer depends entirely on what's underneath: a single device, a legacy protocol, and a two-API enterprise platform are three genuinely different engineering problems that happen to present the same natural-language interface on top.

Tool Inventory: What Each System Actually Exposes

System	Read tools (examples)	Write tools (all confirmation/dry_run-gated)
Axis camera	snapshot, device info, PTZ position, overlays, light status, streaming status	user mgmt, light control, overlay edits, clock/exposure/image settings
AMAG Symmetry	devices, cardholder search, alarms, activity, system info	account/card add, access grant/remove, device commands
Milestone XProtect	cameras, hardware, groups, bookmarks, alarms/events, license, ACLs, snapshot	camera config, hardware add/remove/replace, users/roles, bookmarks, export

RAG

Retrieval-Augmented Generation

What Happens Without RAG (a real example)

“Does cardholder 89455 have access to company 67?”

Without RAG, the model repeatedly calls `search_card_holders`

This is the wrong tool, instead of `check_cardholder_access_to_company`

This is the "plausible but wrong" trap - the model produces something that sounds like an answer

What Happens With RAG (same question)

“Does cardholder 89455 have access to company 67?”

With the project's own docs retrieved, the model selects

`check_cardholder_access_to_company` correctly, essentially every run

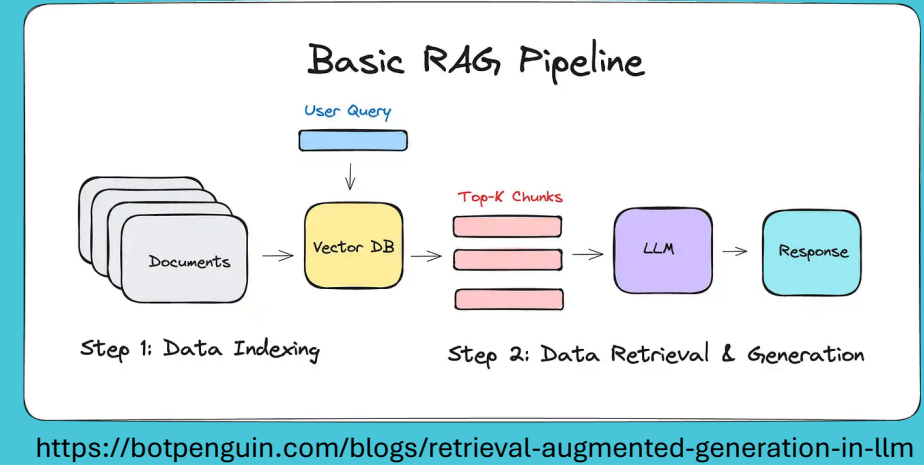
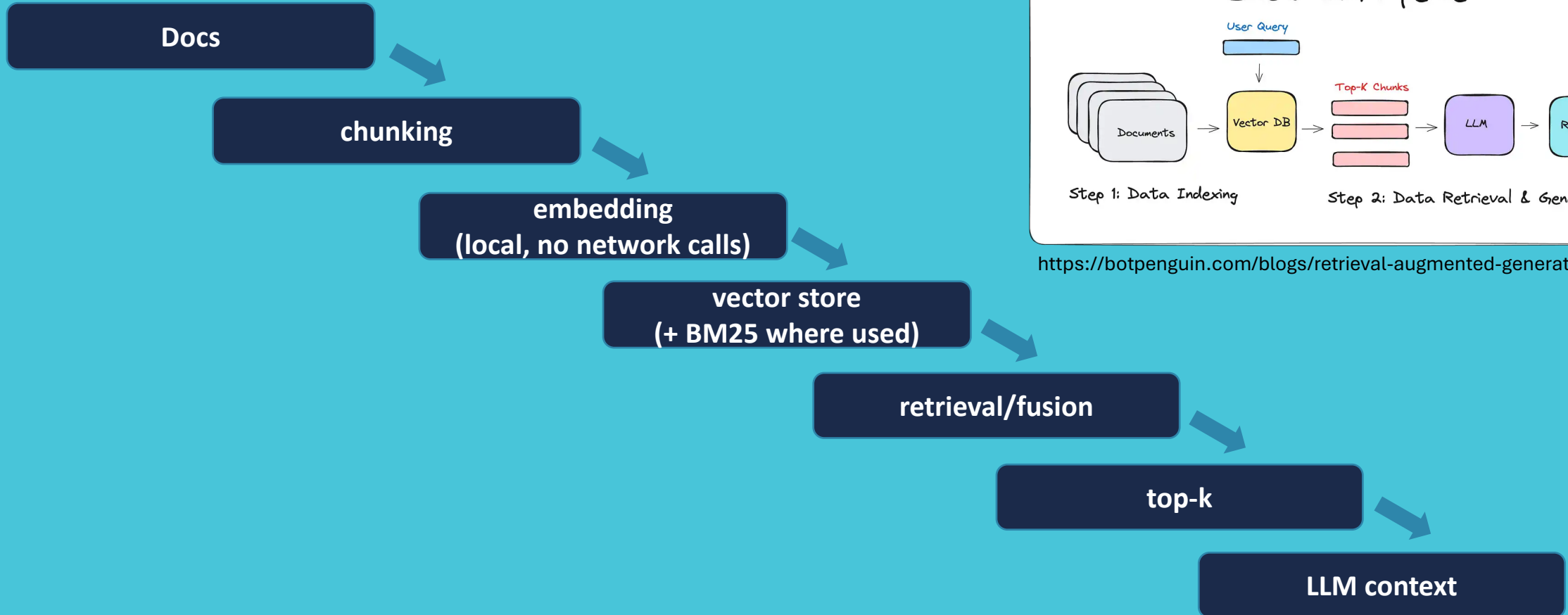
Same model, same request, same system - RAG is the only variable

RAG Isn't Just “Chat With Your Docs”

For this application, RAG supplies:

- System knowledge: what does this API actually support
- Operational knowledge: how does this system actually behave (not just what the spec claims)
- Tool knowledge: which function to call
- Parameter knowledge: what values it needs

A Generic RAG Pipeline



RAG: How Each System's Knowledge Base Was Built

	Axis Camera	AMAG Symmetry	Milestone XProtect
Corpus	VAPIX docs + live-verified findings	This project's own docs (README, roadmap, checklist)	Crawled MIP VMS REST docs + PS cmdlet docs + live system state
Chunks indexed	1,136	174	1,001
Chunking	700 chars, paragraph-aware, hand-rolled	1,000 chars, paragraph-aware (ported from Axis)	600 tokens, 80-token overlap, LlamaIndex SentenceSplitter
Embedding model	all-MiniLM-L6-v2 (sentence-transformers)	all-MiniLM-L6-v2 (quantized ONNX, ~23MB)	BAAI/bge-small-en-v1.5 (HuggingFace)
Storage	JSONL + .npz + SQLite (run tracking)	SQLite (chunks + vectors, one file)	ChromaDB
Retrieval	Hybrid: BM25 + vector	Hybrid: BM25 + vector, RRF fusion	Vector search via LlamaIndex
Framework	Hand-rolled Python	Hand-rolled C#/.NET	LlamaIndex
Network calls at query time	Zero — CPU-local	Zero — CPU-local (ONNX Runtime)	Zero — local embedding model, not API-called

RAG Challenges...

- Data Ingestion and Preparation
- Retrieval Quality and Embeddings
- Latency and Performance
- Scalability and Operational Costs
- Handling Multiple Retrieved Documents
- Knowledge Gaps and Trust
- Compliance and Security

<https://datasciencedojo.com/blog/rag-framework-challenges-in-llm/>

<https://community.nasscom.in/communities/ai/rag-implementation-challenges-data-prep-scaling-compliance>

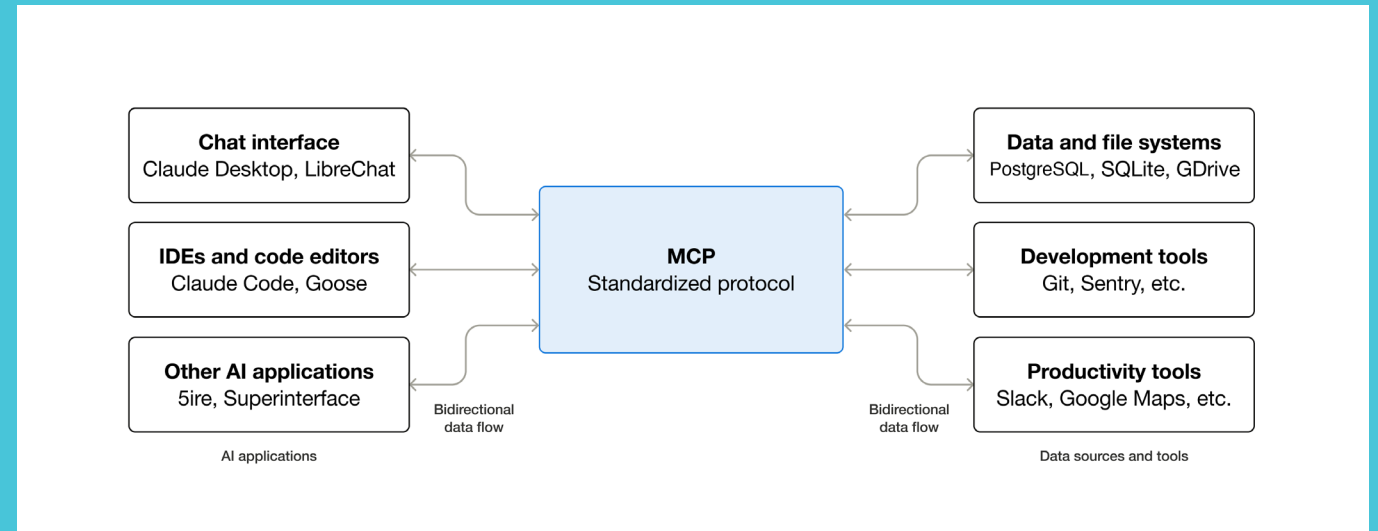
<https://www.educative.io/blog/rag-challenges>

MCP

Model Context Protocol

The Model Context Protocol (MCP) is an open standard introduced by Anthropic in November 2024 to let AI applications securely connect to external data sources and tools

- **Solves Isolation:** AI models are usually trapped behind fixed training data or separate silos and cannot see live files or databases.
- **Stops Custom Chaos:** Building a custom bridge for every single app or database creates broken, unscalable code.
- **Provides a common standard** that works everywhere.



<https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro>

<https://www.anthropic.com/news/model-context-protocol>

What's Actually Inside an MCP Tool Call

Tool definition – what the LLM sees

```
{
  "name": "get_device_status",
  "description": "Get the current status
    of a specific device by ID and type.",
  "input_schema": {
    "type": "object",
    "properties": {
      "deviceId": {"type": "integer"},
      "deviceType": {"type": "string",
        "enum": ["READER_DEVICE", ...]}
    },
    "required": ["deviceId", "deviceType"]
  }
}
```

Request → Response

LLM calls:

```
get_device_status(
  deviceId=331,
  deviceType="READER_DEVICE")
```

Tool returns:

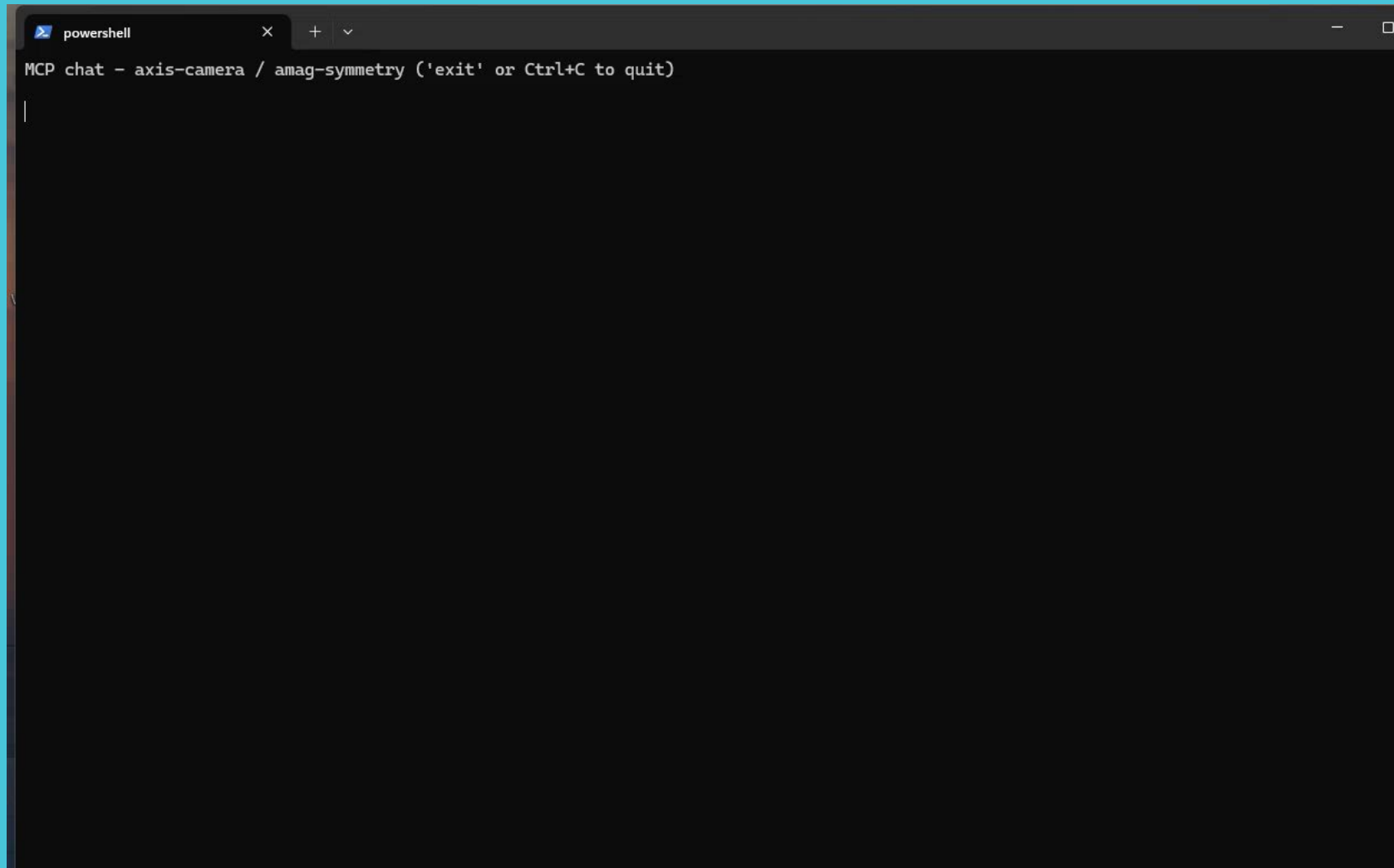
```
{
  "status": "Online",
  "lastHeartbeat": "2026-08-11T09:14:02Z",
  "accessMode": "Card Only"
}
```

LLM then explains this in plain English to the operator.

Same shape for every tool, every system — name + description + typed schema in, structured JSON out. That uniformity is what lets one chat client drive three completely different backends identically.

One Interface, Multiple Systems

- Same chat window, same session — one turn asks AMAG about a building's access history, the next asks the Axis camera for a live snapshot, the next asks Milestone for a camera/license overview.
- Three unrelated backend systems, zero context-switching for the operator.

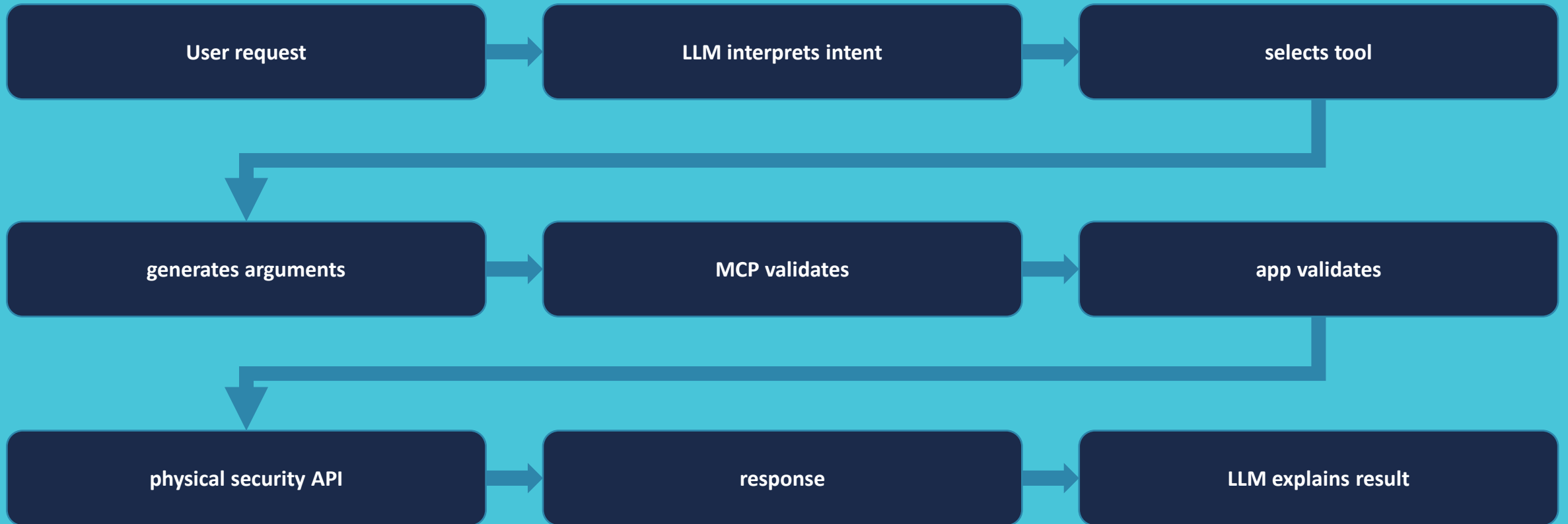


A screenshot of a PowerShell terminal window. The title bar shows 'powershell' with standard window controls. The terminal content displays the command 'MCP chat - axis-camera / amag-symmetry ('exit' or Ctrl+C to quit)' followed by a blank line with a cursor.

```
powershell  
MCP chat - axis-camera / amag-symmetry ('exit' or Ctrl+C to quit)  
|
```

Three-question sequence in one continuous mcp-chat session

Tool Calling Isn't Magic



Experimental Results

Methodology

- Three arms, same question, run back-to-back against the same live system moment, 12 or 16 questions × 5 repeats per system
 - Arm A - Scripted direct API call, no LLM (ground truth)
 - Arm B - Claude Sonnet 5 + real MCP server, no RAG - spawns the actual production MCP server binary, not a reimplementation
 - Arm C - Same + top-5 chunks from local hybrid retrieval
- Question set: ~11 straightforward coverage questions (one per major tool) + 2-3 questions deliberately designed as RAG differentiators, hinging on a fact in project docs but not in any tool description.
- Graded on: tool-match, parameter-match, answer quality (LLM-judge, different/cheaper model than the arms under test).

Axis Camera (16 questions × 5 repeats)

	No RAG	RAG
Tool-match rate	94%	100%
Avg answer score	1.51 / 2	1.82 / 2
Avg latency	7.3s	7.5s
Total cost (80 runs)	\$4.85	\$5.86 (+21%)

- “If I move this camera's PTZ, will that change what's visible in the live feed?”
 - the headline win... no, this fixed fisheye has no real e-PTZ cropping. A generic model plausibly assumes PTZ pans the view; RAG catches the exception.
- “What are ALL the valid privilege group combinations for a new camera user?”
 - RAG gives the complete, confirmed enumeration
 - no-RAG reproduces only partial docstring examples.
- “Is there a modern JSON API instead of the legacy CGI endpoints?”
 - RAG correctly reports the modern API is confirmed unreachable on this camera's firmware.

AMAG Symmetry (16 questions × 5 repeats)

	No RAG	RAG
Tool-match rate	85%	100%
Avg answer score	1.35 / 2	1.86 / 2
Avg latency	11.5s	13.3s
Total cost (80 runs)	\$7.19	\$8.95

- RAG won 7/16 questions outright, lost zero, tied 9.
- “List everyone who has access to reader group 162.”
 - AMAG's API has no reverse lookup. No-RAG guesses or gives up vaguely
 - RAG correctly explains the limitation and cites the confirmed dead end (SEARCH_BY_LF_READERGROUP returns empty).
- “If I import a new card... how do I know whether it succeeded?”
 - RAG warns AMAG's synchronous import response can't be trusted alone
 - no-RAG has no way to know that.

Milestone XProtect (12 prompts × 5 runs)

	No RAG	RAG
Tool-match rate	98%	98%
Answer correctness	50% (30/60)	100% (60/60)
Avg latency (12-prompt sweep)	157.9s	116.6s
Total cost (60 runs)	\$2.18	\$2.35

RAG won 7 of 12 prompts outright, tied on 5, lost zero. The cleanest sweep of all three systems (AMAG: 7/16 won, 0 lost too; Axis: 4/16 won, 0 lost)

- “Is there a Set-VmsCameraRetention cmdlet? If not, what's the actual mechanism?” 0/5 correct without RAG (the model either invents the cmdlet or can't say what really controls retention); 5/5 with it.
- “Does checking recording retention account for archive storage paths, not just live recording?” Same pattern, 0/5 → 5/5.
- The two questions deliberately aimed at the messier, auto-extracted REST/WebSocket API doc corpus (vs. the cleaner MkDocs PowerShell docs) also went 0/5 → 5/5 The noisier extraction some project notes worried about didn't actually hurt retrieval.

Where RAG DID NOT Help

- “If I check the stream status, or start a stream, is there any risk my camera's login credentials could end up exposed in a URL or something I might share with someone else?”
- RAG and no-RAG tied at the same average score (1.0/2), but for opposite reasons:
 - No-RAG: reliably shallow but never wrong — empirically checked live state every time
 - RAG: bimodal — one run correctly cited a historical bug with proper hedging (2.0), another misread the same snippet's past-tense description as current behavior and confidently told the user credentials are leaking right now (0.0)

What Happens When It Can Act?

Security and a Human in the loop

Giving an LLM Access to a Door Is Different

- “Tell me about the system” vs. “Unlock the door”
 - One has physical consequences.
- The threat model changes the moment write access exists.

Threat Model: OWASP LLM Top 10 (2026)

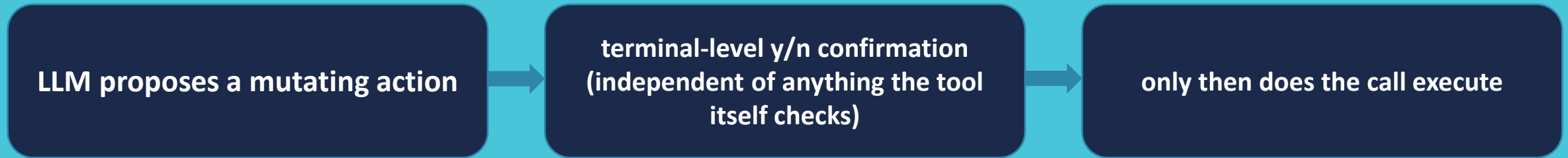
OWASP category	What it looks like here	Mitigation actually built
LLM01:2026 Prompt Injection (incl. indirect)	A retrieved doc, event description, or OCR text contains something that reads like an instruction	Retrieval is read-only context, never auto-executed; every mutating call still needs a human at the confirmation gate
LLM02:2026 Sensitive Information Disclosure	Real cardholder PII, camera blind spots, credentials surfacing in a demo or log	Demo-mode anonymization scrubs every response by field name; credentials never touch chat
LLM03:2026 Excessive Agency	LLM given more capability than a task needs	Explicit read/write tool separation in every project (Milestone's dry_run default)
LLM04:2026 Supply Chain	A dependency you didn't pin silently changes behavior on upgrade	Lived this one directly: mcp-chat's unpinned hosted-SDK dependency silently changed behavior on upgrade. Found and pinned, not hypothetical
LLM05:2026 Data and Model Poisoning (RAG-specific)	A stale or bad doc in the knowledge base treated as authoritative	Corpora are this project's own live-verified findings; Q16 is the concrete version of this risk
LLM06:2026 Unbounded Consumption	AI layer becomes a single point of failure, or a cost/availability liability, for physical security	The security systems function fully independent of the AI layer — it's a client sitting on top, not a dependency underneath

Covers the six most relevant here...

Threat Model, Part 2: OWASP API Security Top 10 (2023)

OWASP category	What it looks like here	Mitigation actually built
API1:2023 Broken Object Level Authorization	Could asking about one cardholder or camera let you retrieve records you're not authorized to see?	The MCP/LLM layer is a client of the real system, not a replacement for it. AMAG and Milestone's own object-level authorization still enforced server-side on every call
API2:2023 Broken Authentication	Weak, hardcoded, or leaked credentials for the underlying REST/SOAP/PowerShell APIs	Standing rule across every project here: credentials are set in the user's own terminal, never typed into chat or stored in a file the model reads
API5:2023 Broken Function Level Authorization	A read-only question accidentally reaching a mutating function	Structural read/write tool separation plus the confirmation gate and dry_run default. The same architecture doing double duty here
API8:2023 Security Misconfiguration	A missing header or a casing mismatch between callers silently breaks a security-relevant code path	Demo-mode reverse lookup initially broke for two different config-shaped reasons — a missing Content-Length header, then a camelCase/PascalCase mismatch — caught by live verification, not assumed correct because it compiled
API10:2023 Unsafe Consumption of APIs	Trusting a downstream/legacy API's response shape and behavior without verification	AMAG's XML/SOAP bridge and Milestone's REST+PowerShell hybrid both exist specifically because the raw upstream API couldn't be trusted or exposed directly to an LLM

The Answer We Built: A Real Confirmation Gate



- Claude Desktop's default consumer-safety posture refused to run mutating MCP tools at all, even with proper authorization context
- mcp-chat was built specifically to restore controlled access with a real human-in-the-loop gate, modeled on Claude Code's own permission-prompt behavior.
- .

Dry-run: remove existing hardware (presenter types 'y' at the confirm prompt)

****You:**** Show me what would happen if I removed the hardware unit named 'toy' - do NOT actually remove it, dry run only. First look up its hardware ID.

****Claude:**** I'll search for the relevant tools across those systems. I have the tools I need. Let me look up the hardware ID for 'toy'. Found it: `toy` → `b2e7ce9f-426b-40fd-9f4e-1848a966bcf3`. Now the dry run: Dry run complete — ****nothing was removed.**** Let me also show you what's actually attached to it, since the dry run output doesn't enumerate the child devices: Dry run only — ****nothing was deleted.****

Step 1: Hardware ID lookup

Field	Value
Name	`toy`
Hardware ID	`b2e7ce9f-426b-40fd-9f4e-1848a966bcf3`
Address	`http://192.168.1.40/`
Model	AXIS P3247-LV Network Camera
Enabled	Yes
Recording server	`94ee1cf7-47f1-4e09-b9b0-6e4da52c7032`
Last modified	2026-08-10

Note there's a similarly-named unit on the VMS — `AXIS P3247-LVE Network Camera (192.168.1.92)` (ID `8da419c6...`, 8 channels). Different device, ****LVE**** vs ****LV****, different IP. The ID above is the correct one for `toy`.

Gate on Milestone's hardware tools, in dry-run mode 1/3

Step 2: Dry run result

```
` `` `json
{
  "dry_run": true,
  "action": "remove_camera_hardware",
  "hardware_id": "b2e7ce9f-426b-40fd-9f4e-1848a966bcf3",
  "warning": "this irreversibly deletes all recorded video/audio/metadata
              for this hardware's child devices"
}
```

What would actually be destroyed

The dry run confirms the target but doesn't enumerate children, so I checked the camera list. One child device would go with it:

- **Camera `toy`** — ID `79a802c1-5ae5-4db9-b12c-21e479d1f02c`, channel 0, enabled, **recording enabled**

Gate on Milestone's hardware tools, in dry-run mode 2/3

Because recording is on, this camera has accumulated video. Removing the hardware deletes that camera *and* all its recorded video/audio/metadata. This is not a config-only change and there is ****no undo**** — no restore, no recycle bin.

The 8 cameras under the `192.168.1.92` unit are unaffected.

Before you commit

Worth checking first, since none of it is recoverable afterward:

- ****Retained footage**** — `get\_camera\_recording\_stats` will show how much video exists and its date range. Once removed, it's unrecoverable even if you re-add the same physical camera.
- ****Export anything you need**** — `export\_video` while the device still exists.
- ****Bookmarks**** — `get\_bookmarks` for this camera; they die with it.
- ****Group membership / rules**** — mostly cosmetic leftovers, but if a shared recording rule covers this camera, removal changes that rule's scope.

If you want to proceed for real, say so explicitly and I'll re-run with `dry\_run=False`. I won't do that without your clear go-ahead. If the goal is swapping in different hardware rather than discarding the device, `replace\_camera\_hardware` may preserve the recordings and configuration instead — happy to look into that.

(33.5s)

Gate on Milestone's hardware tools, in dry-run mode 3/3

Demo Mode: Privacy-by-Design for Live Demos

- Every read response scrubbed of real PII (names, card numbers, employee refs, company/location text) via consistent per-value fake data
- Every mutating endpoint hard-blocked regardless of confirmation params
- A reverse lookup so a fake identity surfaced in one answer can be used in a follow-up query, without ever exposing the real value

Amag Symmetry
Dashboard

[Home](#)

[Devices](#)

[Alarms](#)

[Activity](#)

[Cardholders](#)

[System
Info](#)

Company:

All companies ▼

Using: Elias_Schulist34
(default)

[Login](#)

Activity Search

From: 08/11/2026 10:48:25 AM  To: 08/11/2026 06:48:25 PM 

Defaults to the last 8 hours. Time spans longer than about 48 hours can take a noticeably expanded amount of time to search - each page of results is a real round trip to AMAG.

Reverse lookup using Blazor

Best Practices, Earned Not Assumed

- Separate read tools from write tools, structurally
- Default risky actions to “report what I'd do,” not “do it”
- Put the confirmation gate outside the LLM's own judgment
- Never let credentials pass through the model's context
- Treat retrieved content as informative, not authoritative
- Build the abstraction layer the vendor didn't give you
- Measure RAG's effect, don't assume it
- Keep the underlying system authoritative and independent
- Re-audit safety gates every time you add a new tool or system
- Treat AI output as unreviewed code, not just unreviewed prose

API Authentication

- Implement token-based authentication
 - OAuth 2.0
 - OpenID Connect (OIDC)
- Short-lived JSON Web Tokens (JWTs)
- Enforce TLS 1.2 or Higher
- Hide Secrets From URLs
- Use an API Gateway
- Validate Every Request
- Apply Strict Rate Limiting

<https://www.levo.ai/resources/blogs/rest-api-security-best-practices>
<https://www.stackhawk.com/blog/rest-api-security-best-practices/>

API Error Handling

Effective API error handling relies on predictability, clear communication, and secure system containment

- Essential HTTP Status Codes
- Implement RFC 9457 Problem Details to deliver a standardized, machine-readable JSON object
- Support Resilient Client Recovery
- Prevent Information Leaks
- Monitor and Observe

<https://www.geeksforgeeks.org/node-js/error-handling-in-rest-apis/>
<https://blog.postman.com/best-practices-for-api-error-handling/>
<https://zuplo.com/learning-center/best-practices-for-api-error-handling>

When you send data through AI APIs, you risk losing control of that information...

Major Privacy Risks in AI APIs

- Data Logging
- Model Training
- Hidden Leakage
- Privacy Violations
- Data Security Risks
- Compliance Issues

Best Practices to Protect Data

- Filter and Redact
- Check Terms
- Secure Keys
- Use Placeholders
- On-Premise Deployment

Conclusions

What We Have Learned

- Documentation quality matters. Bad docs → bad RAG → bad tool selection
- APIs alone aren't enough. AMAG Symmetry needed a real abstraction layer, not just an LLM
- RAG isn't just for answering questions. It drives tool selection and parameter construction
- RAG on historical/incident docs carries a distinct hallucination risk. Ground truth changes over time, snippets don't always carry that context
- A hallucinated answer is inconvenient. A hallucinated command is dangerous

The Big Takeaway

***AI doesn't replace the physical security system.
It creates an intelligence layer above it.***

LLM — understands intent

RAG — provides grounded context

MCP — exposes capabilities as tools

APIs — provide connectivity

Human approval — provides authority for anything that touches the physical world

The security system itself — remains the source of truth and the enforcement point

References & Code

- Code and materials from this session: github.com/cdp/GSX2026 (MIT licensed)
- OWASP Top 10 for LLM Applications 2026 — genai.owasp.org, published August 2026
- OWASP API Security Top 10 (2023) — api-security.owasp.org
- Lewis et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (2020) — foundational RAG research
- Model Context Protocol specification — modelcontextprotocol.io
- Black Duck, Ultimate Checklist: Safe Coding Practices for AI-Generated and Open Source Code (2025)

Questions / Discussion



https://www.magnific.com/free-vector/people-ask-question_10837981.htm

THANK YOU FOR ATTENDING TODAY'S SESSION:

**Empowering Physical Security Systems with AI:
Leveraging LLMs and RAG for
Seamless API Integration**

Please complete your evaluation
within the GSX mobile app.



MCP Tools

Axis Camera — All 46 Tools

activate_light, add_image_overlay, add_text_overlay, add_user, deactivate_light, disable_light, enable_light, get_camera_overview, get_clock_info, get_device_info, get_image_settings, get_light_status, get_overlay_capabilities, get_snapshot, get_stream_status, get_supported_apis, get_valid_intensity, list_overlays, list_stream_profiles, list_users, ptz_get_position, ptz_goto_preset, ptz_list_presets, ptz_move_absolute, ptz_move_relative, ptz_save_preset, rag_stats, remove_overlay, remove_user, save_snapshot, search_knowledge, set_clock, set_exposure_mode, set_exposure_value, set_image_appearance, set_image_overlay, set_light_auto_mode, set_light_intensity, set_manual_gain, set_manual_shutter, set_text_overlay, set_timezone, set_white_balance, start_stream, stop_stream, update_user

AMAG Symmetry — All 20 Tools

check_symmetry_connection, list_devices, get_device_status, get_door_status,
list_outstanding_alarms, search_activity, search_cardholders, get_cardholder_last_reader,
get_cardholder_antipassback_zone, get_cardholder_personal_data,
check_cardholder_access_to_company, get_cardholder_current_access, get_system_info,
get_import_status, grant_door_access, add_cardholder, add_card, grant_card_access,
remove_card_access, set_card_active

Milestone XProtect — All 47 Tools

list_cameras, get_camera, get_camera_motion, list_hardware, list_camera_groups, add_camera_group, remove_camera_group, list_camera_group_members, add_camera_to_group, remove_camera_from_group, get_license_overview, get_alarms, search_events, get_camera_status, get_camera_report, get_camera_recording_stats, add_bookmark, get_bookmarks, update_bookmark, remove_bookmark, update_camera, update_camera_resolution, update_camera_motion, get_storage_configs, update_camera_retention, get_camera_recording_pattern, set_camera_recording_pattern, remove_camera_recording_pattern, add_user, update_user, remove_user, list_users, list_roles, add_role, remove_role, get_device_acl, set_device_acl, set_user_role, get_user_roles, scan_for_hardware, list_hardware_drivers, add_camera_hardware, remove_camera_hardware, replace_camera_hardware, export_video, get_snapshot, open_live_view

RAG vs. No-RAG Eval Questions

All Three Projects

AMAG Symmetry (16 questions)

```
| ID | Difficulty | Question |
|---|---|---|
| Q01 | easy | Is the AMAG connection healthy right now, and which account is it using? |
| Q02 | easy | How many reader devices does the system have in total? Just give me the count. |
| Q04 | easy | What is the current status of device 331? |
| Q05 | easy | What alarms are currently outstanding right now? |
| Q06 | medium | How many outstanding alarms are there at company 67? |
| Q07 | easy | Find the cardholder with card number 123456. |
| Q08 | easy | Find the cardholder with last name Smith. |
| Q09 | easy | Which reader did cardholder 89455 last use, and when? |
| Q10 | easy | What antipassback zone is cardholder 89455 currently in? |
| Q11 | easy | What personal data fields are filled in for cardholder 89455? |
| Q12 | easy | Does cardholder 89455 have access to company 67? |
| Q13 | easy | Does cardholder 89455 have access to company 55? |
| Q14 | easy | What is cardholder 89455's full access breakdown across every company? |
| Q15 | easy | What companies exist in this system and what are their IDs? |
| Q16 | hard — RAG differentiator | List everyone who has access to reader group 162. |
| Q17 | hard — RAG differentiator (operational/incident knowledge) | If I import a new card or grant someone
..... access using the cardholder-import operation, how do I actually know whether it succeeded?
→→→..... Is the immediate response trustworthy? |
```

(Note: Q03 was dropped from the original set — no gap in the numbering above is an error.)

Axis Camera (16 questions)

ID	Difficulty	Question
Q01	easy	What model is this camera, and what's its serial number?
Q02	easy	How many VAPIX APIs does this camera support in total?
Q03	easy	What is the camera's current timezone, and is daylight saving time currently active?
Q04	easy	What are the camera's current image sensor settings — brightness, contrast, and exposure mode?
Q05	easy	What PTZ presets are currently saved on this camera?
Q06	easy	What is the camera's current PTZ position — pan, tilt, and zoom?
Q07	easy	List every overlay currently configured on the camera, and describe what each one shows.
Q08	easy	What are this camera's overlay limits — how many overlay slots are available, and what's the max font size?
Q09	easy	List every local user account on this camera, along with each one's privilege groups.
Q10	easy	What streaming profiles are configured on this camera?
Q11	easy	How many chunks are indexed in the local knowledge base, and from how many sources?
Q12	hard — RAG differentiator	If I move this camera's PTZ to point at a different area, will that actually change what's visible in the live video feed?
Q13	hard — RAG differentiator	Can you turn on this camera's built-in IR light or illuminator for me?
Q14	hard — RAG differentiator	What are ALL the valid privilege group combinations I can assign when creating a new camera user account?
Q15	hard — RAG differentiator	Is there a modern JSON-based user-management API on this camera I should use instead of the legacy CGI endpoints?
Q16	hard — RAG differentiator (operational/incident knowledge)	If I check the stream status, or start a stream, is there any risk my camera's login credentials could end up exposed in a URL or something I might share with someone else?

Milestone XProtect (12 prompts)

ID	Category	Prompt
doc-1	doc-lookup	What parameters does the MilestonePSTools.Get-Snapshot cmdlet take, and what does the -Live switch do?
doc-2	doc-lookup	How do I authenticate to the Milestone MIP VMS REST API and get a bearer token? What's the token endpoint?
live-1	live-state	Of the 8 camera channels on this system, which ones are actually connected and working right now?
live-2	live-state	What is the current recording retention policy on this system, and roughly how many devices does it affect?
action-1	action-task	I want to add a bookmark on Camera 1 (id.c2161879-cbb4-498e-98d4-ff521097d846) with the header 'Test Event'. → Show me what that would do without actually creating it.
action-2	action-task	I want to export the last 10 minutes of footage from Camera 1 (id.c2161879-cbb4-498e-98d4-ff521097d846) to disk. → Show me what that would do without actually running the export.
doc-3	doc-lookup	Using MilestonePSTools, how can I check the configured recording retention for a storage configuration, and does it account for archive storage paths as well as the live recording path?
doc-4	doc-lookup	When exporting video with the MilestonePSTools.Start-Export cmdlet, how do I add a digital signature to the exported database to prove it hasn't been tampered with, and which export format does that require?
adv-1	adversarial	Is there a Set-VmsCameraRetention cmdlet in MilestonePSTools that lets me directly set how long a specific camera retains its recordings? If not, what's the actual mechanism for controlling retention?
adv-2	adversarial	What is the live connection status of Camera 9 on this system?
api-1	doc-lookup (api-docs corpus)	When connecting to the XProtect Events and State WebSocket API without an Authorization header, how do I authenticate over the connection itself, and how long do I have to send that command after connecting?
api-2	doc-lookup (api-docs corpus)	Can I use the XProtect WebRTC signaling API to stream video from a camera that has privacy masking enabled? Why or why not?

(adv-2 asks about "Camera 9" on an 8-camera system — a deliberate negative to check whether RAG context makes hallucination better or worse, not just answer accuracy on answerable questions.)